

Questions and Answers:

1. How many entries do you have in your database who have applied for Fall 2026?

Answer: 33081

SQL: SELECT COUNT(p_id) AS term_count FROM applicants WHERE term = 'Fall 2026';

Reasoning: This is about as simple a SQL query can get. Counts the IDs in applicants that are from Fall 2026.

2. What percentage of entries are from international students (not American or Other) (to two decimal places)?

Answer: 46.68

SQL: SELECT ROUND(

```
(
    (SELECT CAST(COUNT(us_or_international) AS NUMERIC) FROM applicants
    WHERE us_or_international = 'International')
/
    (SELECT CAST(COUNT(us_or_international) AS NUMERIC) FROM applicants)
)
* 100, 2);
```

Reasoning: To put everything into one query requires subqueries. In this case, I needed the overall count and the limited count of “International” applicants. Those two numbers are then divided and multiplied by 100 to get a percentage, then the whole thing is rounded to two places.

3. What is the average GPA, GRE, GRE V, GRE AW of applicants who provide these metrics?

Answer: GPA: 3.770083107191303, GRE: 261.6066312997348, GRE V: 161.18610579987254, GRE AW: 9.174500851788785

SQL: SELECT

```
(SELECT AVG(gpa) FROM applicants WHERE gpa > 0) AS avg_gpa,
(SELECT AVG(gre) FROM applicants WHERE gre > 0) AS avg_gre,
(SELECT AVG(gre_v) FROM applicants WHERE gre_v > 0) AS avg_gre_v,
(SELECT AVG(gre_aw) FROM applicants WHERE gre_aw >= 0) AS avg_gre_aw;
```

Reasoning: One select per average needed. Each individual query is nothing special. I opted to fill fields not entered here with -1 (mostly so it would be very obvious if I forgot to check when I looked at any aggregate), so each subquery needs to make sure it only selects applicants that left that item. Then wrapped the whole thing in one select to return the whole row of data in one go instead of calling each individually.

4. What is the average GPA of American students in Fall 2026?

Answer: 3.773936950591131

SQL: SELECT AVG(gpa) FROM applicants WHERE us_or_international = 'American' AND gpa > 0;

Reasoning: This is also a pretty standard query. Selected the average of the GPA column, and I added a check to make sure we didn’t grab any applicants with no GPA entry (again, defaulted to blank entry being -1).

5. What percent of entries for Fall 2026 are Acceptances (to two decimal places)?

Answer: 35.84

SQL: SELECT ROUND(

```
(
    (SELECT CAST(COUNT(p_id) AS NUMERIC) FROM applicants WHERE status =
'Accepted' AND term = 'Fall 2026')
/
```

(SELECT CAST(COUNT(p_id) AS NUMERIC) FROM applicants WHERE term = 'Fall 2026')
)
* 100, 2);

Reasoning: Same process as question 2, requiring subqueries. I needed the total and the subtotal (Acceptances and total entries) to calculate the percentage. Each subquery limited to the specific term I needed.

6. What is the average GPA of applicants who applied for Fall 2026 who are Acceptances?

Answer: 3.7884931096465286

SQL: SELECT AVG(gpa) FROM applicants WHERE term = 'Fall 2026' AND gpa > 0;

Reasoning: Also a pretty standard query. Just need to make sure the applicant left the value (-1 still being the “not entered” response).

7. How many entries are from applicants who applied to JHU for a masters degrees in Computer Science?

Answer: 37

SQL: SELECT count(p_id) FROM applicants

WHERE program LIKE '%Computer Science%' AND university LIKE '%Johns Hopkins%';

Reasoning: Mostly standard query. I opted for a similarity search to try and decrease the odds of missing some hits due to bad user input (which still happened). Using LIKE allows for wildcards to try and get any common mistakes.

8. How many entries from 2026 are acceptances from applicants who applied to Georgetown University, MIT, Stanford University, or Carnegie Mellon University for a PhD in Computer Science?

Answer: 155

SQL: SELECT COUNT(p_id)

FROM applicants

WHERE extract(YEAR FROM date_added) = 2026 AND

program LIKE '%Computer Science%' AND

university LIKE ANY (ARRAY['%Georgetown University%', '%Massachusetts Institute of Technology%', '%MIT%', '%Stanford University%', '%Carnegie Mellon%']);

Reasoning: The question left a little room for interpretation, so I assumed “entries from 2026” meant any entries that were submitted in 2026. Therefore, in the WHERE I extracted the year from the data it was added and only selected if it was in 2026. Then I searched program like the previous question. For the university list, I opted to use an array of the universities with the same LIKE system, but I also added MIT on its own, because this is an incredibly common abbreviation.

9. Do your numbers for question 8 change if you use LLM Generated Fields (rather than your downloaded fields)?

Answer: Yes (156)

SQL: SELECT COUNT(p_id)

FROM applicants

WHERE extract(YEAR FROM date_added) = 2026 AND

llm_generated_program LIKE '%Computer Science%' AND

llm_generated_university LIKE ANY (ARRAY['%Georgetown University%', '%Massachusetts Institute of Technology%', '%MIT%', '%Stanford University%', '%Carnegie Mellon%'])

Reasoning: Same query as 8, but with the llm_generated columns in place of the standard. I currently have 156, but that is after running the database update (all of these numbers are, I didn't want to drop the whole database to run the data_query again). I think the number was a couple bigger before the update, but this is the data in front of me right now.

10. How many GRE AW scores reported exceeded the maximum score attainable?

Answer: 153

SQL: `SELECT COUNT(p_id) FROM applicants WHERE gre_aw > 6;`

Reasoning: I saw the GRE AW average score from question 3 and I thought it looked questionable. Checking the GRE website, it was easy to find that the GRE AW is scored from 0 to 6 and an average of 9 was significantly over that maximum. I pulled the data to see if this was a consistent data issue, or if there were some outliers that were significantly skewing the data (someone entering the GRE score in the GRE AW field by accident, for example). This would warrant more research if I was doing a serious data analysis for a customer.

11. How many GRE scores reported exceeded the maximum score attainable?

Answer: 13

SQL: `SELECT COUNT(p_id) FROM applicants WHERE gre > 340`

Reasoning: The previous question made me wonder if this was also an issue with normal GRE scores. There are some over inflated responses, but not enough that I think it would be statistically significant for this particular column. If we keep working with this data, I will probably remove items that are “out of range” for the tests they’re supposed to reflect.

Limitations

Most of the analytic responses did not surprise me. When dealing with data in general one of the guiding principles is, “Garbage in, garbage out”. Analysis of data is only as good as the data that the analysis is made on. There are a lot of things about this data set that make its use beyond a purely academic study questionable. When data is self-reported, the question of the motivation of the respondent needs to be considered. There are certainly many reasons for an individual to lie about certain pieces of information (like GPA), to somehow make themselves feel better or to make other people reading the entry think better of them.

Any data that comes in from an open-ended field will lack standardization (as is the case with our Program and University fields). This is made worse by having fields that should have some kind of bounds that fail to do bounds checking. The example with the GRE AW is a perfect example. It would be relatively easy to do a bounds check and refuse submissions that are simply not possible. From the data on hand, it is pretty obvious this isn’t done.